AI 기반 자율주행 로봇 개발 실습 과정

cchyun@gmail.com

강사 소개 (현청천)

🎓 주요이력

- **현) 주식회사 위드알엘 대표이사**
- **현) 모두의연구소 반려로봇 랩장**
- 전) 텔레픽스 AI 사업팀 이사
- 전) 헬로엔엠에스 대표
- 전) 삼성전자 선임연구원

🏆 수상경력

- **2021.07** 인공지능 그랜드챌린지 입상
- **2019.12** 성남시 빅데이터센터 아이디어톤 **우승**
- **2019.07** AI Starthon
  - Intent 2위 / Query 3위 / Movie 4위
- **2018.12** 네이버 NLP Challenge NER **장려상**

🔬 연구 분야

- 인공지능, 강화학습 기반의 반려로봇 제작

✉️ cchyun@gmail.com
🏠 paul-hyun.github.io | 📝 with-rl.tistory.com | 🌐 with-rl.com | 📺 @with-rl

P1 · Python 기초 — Day 1~3 (18h)

- **Day 1 — Python 환경 & 기초 문법**
  - Git 기초 (init, commit, push, branch) / 개발 환경 설정 (venv, pip, VSCode)
  - 자료형(int·float·str·list·dict·tuple)·제어문·함수·람다·예외 처리
  - [실습] 계산기 CLI 프로그램 (사칙연산·예외 처리·모듈 분리)
- **Day 2 — Python OOP & 데이터 처리**
  - 클래스·상속·추상화·덕 타이핑 / `__init__`, `__repr__`, `@property` 활용
  - NumPy 배열 연산 (브로드캐스팅·슬라이싱) & Matplotlib 다중 플롯 시각화
  - [실습] 센서 데이터 시뮬레이션 (가우시안 노이즈 생성·이동평균 필터·플롯)
- **Day 3 — Python 실전**
  - 파일 I/O (CSV·JSON 읽기/쓰기) / 정규표현식으로 로그 패턴 추출
  - 멀티스레딩 (threading.Thread·Lock) & 멀티프로세스 (multiprocessing.Pool) 기초
  - [실습] 센서 로그 파싱 & 시각화 자동화 (배치 처리·PNG 자동 저장)

P2 · 비전·센서 퓨전 — Day 4~5 (12h)

- **Day 4 — OpenCV 입문 & 기초 영상처리**
  - OpenCV 설치·기초: `imread`, `imshow`, 색공간 변환 (BGR↔HSV·GRAY)
  - 필터링 (GaussianBlur·MedianBlur) & 엣지 검출 (Canny, Sobel, Laplacian)
  - 모폴로지 연산 (erosion·dilation·opening·closing) & 컨투어 검출
  - [실습] 실시간 웹캠 에지 검출 파이프라인 (trackbar로 파라미터 실시간 조정)
- **Day 5 — 카메라 캘리브레이션 & YOLO & Kalman 개요**
  - 핀홀 카메라 모델 / 내부 파라미터(fx·fy·cx·cy)·왜곡 계수 / 체커보드 캘리브레이션
  - YOLOv8 추론: Ultralytics API, 사전 학습 모델 로드·Confidence·NMS 설정
  - Kalman Filter 개념 개요: 예측-갱신 사이클, 노이즈 공분산 Q·R (이론)
  - [실습] 로봇 카메라 실시간 객체 탐지 & BBox 좌표 콘솔 출력

P3 · ROS2 기초 — Day 6~8 (18h)

- **Day 6 — ROS2 설치 & 핵심 개념**
  - ROS2 Humble 설치 & `~/.bashrc` 환경설정 (`source /opt/ros/humble/setup.bash`)
  - 노드(Node)·토픽(Topic)·서비스(Service)·액션(Action) 개념 / colcon 빌드 시스템
  - `ros2 topic list/echo/pub`, `ros2 node info`, `ros2 service call` CLI 활용
  - [실습] Hello ROS2 노드 작성 & `ros2 run`으로 실행 확인
- **Day 7 — Publisher / Subscriber & 서비스**
  - Python rclpy Publisher / Subscriber 구현 (`create_publisher`, `create_subscription`)
  - 서비스 서버/클라이언트 구현 (`create_service`, `create_client`) & 비동기 호출
  - 액션 서버/클라이언트 구현 (Goal·Feedback·Result 패턴) 기초
  - [실습] 카메라 이미지 토픽 퍼블리시 (`sensor_msgs/Image`) & RViz2 확인
- **Day 8 — 런치 파일 & 파라미터 & rqt**
  - Python 런치 파일 작성 (`launch.py`) & 파라미터 YAML 파일 관리
  - 커스텀 메시지(`msg`) / 서비스(`srv`) / 액션(`action`) 타입 정의 & colcon 빌드
  - [실습] rqt_graph·rqt_plot·rqt_console로 노드 토폴로지·데이터·로그 디버깅

P4 · TF2·Gazebo — Day 9~10 (12h)

- **Day 9 — TF2 좌표 변환 & 센서 토픽 & rosbag2**
  - TF2 아키텍처: `static_transform_broadcaster`, `TransformBroadcaster` / `TransformListener`
  - LiDAR (`sensor_msgs/LaserScan`) · IMU (`sensor_msgs/Imu`) · Camera 토픽 파이프라인
  - rosbag2 녹화 (`ros2 bag record`) & 재생 (`ros2 bag play`) / 데이터 오프라인 분석
  - [실습] TurtleBot3 TF 트리 시각화 (`tf2_tools view_frames` → PDF 확인)
- **Day 10 — Gazebo 시뮬레이션**
  - URDF/SDF 로봇 모델링: 링크·조인트·관성·충돌 박스 작성
  - 시뮬레이션 플러그인: IMU(`imu_plugin`), LiDAR(`ray_plugin`), Camera(`camera_plugin`)
  - `robot_state_publisher` · `joint_state_publisher` 연동 & RViz2 URDF 시각화
  - [실습] 차동 구동 로봇 Gazebo 런치 & `/cmd_vel` 토픽으로 키보드 원격 조종

P5 · SLAM·AMCL — Day 11~12 (12h)

- **Day 11 — SLAM 이론 & SLAM Toolbox 실습**
  - SLAM 문제 정의: 동시 지도 작성(Mapping)·위치 추정(Localization) 결합 과제
  - SLAM 분류: Filter-based (EKF·PF) vs Graph-based (Pose Graph) 개념 개요
  - SLAM Toolbox 구조: `async_slam_toolbox_node` / 온라인 매핑 파라미터 설정
  - 맵 품질 평가 기준 & `map_saver_cli`로 `.pgm`+`.yaml` 저장
  - [실습] TurtleBot3 Gazebo 가상 환경 매핑 (teleop으로 주행 → 지도 완성)
- **Day 12 — AMCL 로컬라이제이션**
  - 파티클 필터(PF) 원리 & MCL(Monte Carlo Localization) 알고리즘 단계별 이해
  - AMCL 핵심 파라미터: `min/max_particles`, `laser_z_hit`, `odom_model_type`
  - 글로벌 로컬라이제이션 (`global_localization` 서비스) & 초기 포즈 설정
  - [실습] 사전 저장 지도 로드 → AMCL 위치 추정 (Gazebo) & RViz2 파티클 확인

P6 · Nav2 — Day 13~14 (12h)

- **Day 13 — Nav2 아키텍처 & 플래너**
  - Nav2 전체 스택 구조: BT Navigator · Planner Server · Controller Server · Costmap2D
  - Costmap2D 레이어 구성: Static Layer · Obstacle Layer · Inflation Layer 파라미터
  - 글로벌 플래너: NavFn (Dijkstra/A*), Smac Planner (SE2·Hybrid-A*) 비교
  - DWB 로컬 플래너: 속도 샘플링·critic 점수 기반 최적 경로 선택 원리
  - [실습] Nav2 기본 자율 주행 (Gazebo) — RViz2에서 Goal Pose 전송 & 주행 확인
- **Day 14 — Nav2 장애물 회피 & 파라미터 튜닝**
  - 동적 장애물 처리: Obstacle Layer 갱신 주기 & `clearing_enabled` 설정
  - DWB 파라미터 튜닝: `max_vel_x`, `acc_lim_x`, `PathAlign/GoalAlign` critic 가중치
  - 목적지 지정 반복 주행 테스트: Python `BasicNavigator` API 활용 Waypoint 순회
  - [실습] 좁은 복도 장애물 회피 (Gazebo) — inflation_radius 조정으로 통과 성공

P7 · PID·A*/RRT* — Day 15 (6h)

- **PID 이론 & Ziegler-Nichols 튜닝**
  - 비례(P)·적분(I)·미분(D) 항 역할 / 정상 상태 오차·오버슈트·진동 관계
  - Ziegler-Nichols 임계 이득법 튜닝 절차 (Ku, Tu → Kp, Ki, Kd 도출)
  - [실습] Python으로 1차 시스템 PID 시뮬레이션 구현 & Matplotlib 반응 곡선 시각화
- **경로 계획 이론 개요**
  - A* 알고리즘: 휴리스틱 함수 h(n) 설계 (맨해튼·유클리드) & f(n)=g(n)+h(n)
  - RRT*: 확률적 트리 확장 & 재연결(Rewire)로 최적 경로 수렴 원리
  - Nav2 플래너와의 연결: NavFn(A* 기반) / Smac(Hybrid-A*/RRT* 기반) 대응 관계 설명
  - PID 속도 제어 + A*/RRT* 경로 계획 통합 구조 이해

P8 · 터틀봇 캡스톤 (1/2) — Day 16~18 (18h)

- **Day 16 — TurtleBot3 세팅 & 아키텍처 설계**
  - 로봇 안전 교육: 비상 정지(E-Stop) 절차·충돌 방지 수칙·배터리 관리
  - TurtleBot3 실물 하드웨어 구성 확인: OpenCR·LiDAR·Raspberry Pi·IMU
  - SSH 접속·ROS2 통신 확인 (`ros2 topic list`, `ros2 node list`)
  - 캡스톤 최종 미션 시나리오 발표 & 팀 아키텍처 설계·역할 분담
- **Day 17 — SLAM 매핑 & YOLO 연동**
  - SLAM Toolbox로 실물 환경 매핑 (teleop 주행 → 지도 완성 → `map_saver_cli`)
  - 카메라 캘리브레이션 (실물 체커보드 촬영 → 왜곡 계수 산출)
  - YOLO 추론 결과 → `vision_msgs/Detection2DArray` ROS2 토픽 퍼블리시
  - TF2 좌표 변환 통합: 카메라 프레임 → base_link → map 좌표 변환 & RViz2 시각화
- **Day 18 — Nav2 + 객체 회피 통합 1**
  - YOLO 탐지 BBox → 3D 포인트 역투영 → Nav2 Costmap Obstacle Layer 실시간 반영
  - 동적 장애물 회피 파이프라인 구축 (탐지→비용맵 갱신→재계획 루프)
  - [실습] 1차 통합 테스트: 탐지된 사람 앞 정지 & 우회 경로 재계획 확인·디버깅

P8 · 터틀봇 캡스톤 (2/2) — Day 19~20 (12h)

- **Day 19 — Nav2 + 객체 회피 통합 2 & 튜닝**
  - AMCL 로컬라이제이션 실물 안정성 검증: 파티클 수·레이저 파라미터 조정
  - 전체 파이프라인 반복 주행 테스트: 출발→웨이포인트 순회→귀환 시나리오
  - PID(선속도·각속도) & DWB critic 가중치 Fine-Tuning으로 안정 주행 달성
  - 예외 처리 보강: 로컬라이제이션 실패 복구·경로 재계획 실패 시 안전 정지
- **Day 20 — 전체 통합 & 최종 발표**
  - 전체 미션 시나리오 반복 주행 & 시연 영상 녹화 (최소 3회 성공 기준)
  - 팀/개인별 프로젝트 발표 (15분): 아키텍처·구현·시연·트러블슈팅 공유
  - Q&A·강사 피드백·동료 리뷰 / 개선점 정리
  - 전체 회고 & 향후 학습 로드맵 안내 (ROS2 심화·강화학습·실제 로봇 배포)